

Adaptive KV Cache Reuse for Fast Long-Context LLM Serving

Fei Li

School of Computer Science and
Technology, Xi'an Jiaotong University
Xi'an, China
lifei@stu.xjtu.edu.cn

Song Liu

School of Computer Science and
Technology, Xi'an Jiaotong University
Xi'an, China
liusong@mail.xjtu.edu.cn

Yan Liu

School of Computer Science and
Technology, Xi'an Jiaotong University
Xi'an, China
gmliuyan@stu.xjtu.edu.cn

Jinhua Cui

School of Computer Science and
Technology, Huazhong University of
Science and Technology
Wuhan, China
csjhcui@gmail.com

Shiqiang Nie

School of Computer Science and
Technology, Xi'an Jiaotong University
Xi'an, China
shiqiang.nie@xjtu.edu.cn

Jinyu Wang

School of Computer Science and
Technology, Xi'an Jiaotong University
Xi'an, China
jinyu.wang@xjtu.edu.cn

Weiguo Wu

School of Computer Science and
Technology, Xi'an Jiaotong University
Xi'an, China
wguwu@xjtu.edu.cn

Abstract

In long-context Large Language Model (LLM) inference, the Time-To-First-Token (TTFT) latency incurred by the prefill stage has become the foremost bottleneck limiting interactive performance and deployment cost. KV Cache reuse offers a direct path to reduce redundant prefill, yet traditional prefix caching applies only to strict-prefix scenarios; directly reusing KV Cache in non-prefix settings breaks the cross-chunk global attention relationships and causes significant degradation in generation quality. When reusable KV Cache is offloaded to GPU-external cache pools, I/O overheads across heterogeneous hardware tiers further emerge as a new TTFT bottleneck. Efficient non-prefix KV Cache reuse therefore requires both semantic-consistency recovery and compute-I/O co-optimization.

This paper presents CacheTune, a frequency-guided and hardware-aware KV Cache reuse system for long-context LLM serving. CacheTune first identifies, offline, the KV pairs most critical to cross-attention recovery through frequency-domain analysis, and then selectively recomputes only these semantic-critical tokens online while reusing the remaining KVs. To turn this semantic selection into end-to-end latency reduction, CacheTune further combines sparse KV transfer, multi-stream asynchronous overlap, deferred positional-encoding recovery, and hardware-aware adaptive recomputation-ratio tuning to balance computation and data movement across heterogeneous cache pools. Evaluations on mainstream LLMs and long-context tasks show that CacheTune achieves $3.72\times\text{--}4.86\times$ TTFT speedup and $3.93\times\text{--}6.21\times$ higher throughput while maintaining generation quality close to

full recompute. Even when caches are offloaded to I/O-bound SSD/HDD storage, CacheTune sustains $2.34\times\text{--}2.36\times$ TTFT speedup through adaptive recomputation.

1 Introduction

Large Language Models (LLMs) are progressively evolving from short-dialogue generation toward complex applications such as retrieval-augmented generation (RAG) [17], agentic reasoning [38], long-document question answering [2], and multi-turn conversation summarization [4], with the context size carried by inference requests rapidly growing. In these scenarios, the input is no longer a single contiguous short prompt but a long context assembled by concatenating multiple retrieved documents, dialogue histories, system instructions, and reusable knowledge fragments. For such requests, the system bottleneck is no longer the per-token decoding stage; rather, it is dominated by the Time-To-First-Token (TTFT) latency produced by long-context prefill [6, 37]. Although inference frameworks such as vLLM substantially improve KV Cache memory management through mechanisms like PagedAttention [15], they still need to repeatedly perform forward computation over a large number of recurring, theoretically reusable context fragments, leading to severe compute waste and interactive latency.

To reduce the overhead caused by redundant prefill, a widely adopted line of recent work is to precompute KV Cache for reusable text segments and directly reuse them in subsequent requests [22, 39, 40]. However, these approaches face a fundamental challenge: the reusable content is usually not a strict prefix of the final prompt; instead, it more

commonly manifests as multiple mutually independent document chunks, retrieved blocks, or historical segments that are dynamically concatenated into the current query. If these chunks are independently encoded into the local KV Cache, each chunk’s representation lacks the cross-chunk attention relationships that should have been established in the global context. We refer to such inter-chunk attention as **cross-attention**. Consequently, a discrepancy arises between the independently encoded caches and the true representation under the fully concatenated prompt, and this discrepancy propagates directly into the model’s attention distribution and final generation. Meanwhile, moving KV Cache across hardware tiers incurs non-negligible overheads in long-context serving [5, 16, 26], elevating KV Cache reuse from a seemingly storage-only problem into a joint optimization problem spanning semantic recovery and system scheduling.

Existing studies have shown that KV Cache reuse in non-prefix scenarios cannot be accomplished through simple cache concatenation; rather, the broken global semantic dependencies and positional relationships must be explicitly handled [1, 21, 37]. However, two key questions remain unresolved. As illustrated in Figure 1, first, existing methods primarily focus on chunk boundaries, positional compensation, or local repair with fixed structures [10, 35, 37], but lack an explicit representation of which tokens truly bridge the semantic gap between independent and global encoding. As a result, these methods may not accurately cover the semantically critical points distributed throughout the entire context. If one could more precisely identify the key tokens that carry global semantic connections and most warrant recomputation, the missing information could potentially be restored at extremely low recomputation cost. Second, when considering large-scale, realistic deployment environments where the total volume of reusable KV Cache keeps growing, the storage medium will inevitably extend from GPU/CPU memory to disks, and even to data-center-scale caching systems [20, 23, 32]. Under such conditions, what governs system efficiency is the dynamic balance between recomputation cost and cross-tier KV Cache transfer cost. Ignoring the deep coupling between I/O and computation [24, 41], even methods that theoretically reduce redundant computation may, in practice, be offset by lengthy data-transfer paths and high movement overhead, failing to translate into real end-to-end latency gains.

Motivated by these observations, we present CacheTune, an algorithm–system co-design framework for efficient non-prefix KV Cache reuse in LLM serving. CacheTune uses a one-time offline frequency-domain analysis to identify the tokens most critical to global semantic recovery, turning recomputation from a coarse latency cost into a targeted repair. To hide cross-tier I/O overhead, CacheTune overlaps sparse cache transfer, online recomputation, and deferred positional-encoding recovery through a multi-CUDA-stream

asynchronous pipeline. Built on this pipeline, a hardware-aware recomputation-ratio search adaptively balances computation and data movement across heterogeneous storage tiers. As a result, CacheTune enables high-quality non-prefix KV reuse with near-full-recompute accuracy while substantially reducing TTFT across diverse storage configurations.

The contributions of this paper are summarized as follows:

- We propose a frequency-guided KV Cache importance modeling and selective recomputation method. It identifies the KVs most critical to global semantic recovery and effectively restores the cross-attention missing in non-prefix reuse at low recomputation cost.
- We design an index-aware KV offloading and sparse reuse pipeline. By transferring only the non-recomputed KVs and asynchronously overlapping sparse transfer, online recomputation, and deferred positional-encoding recovery, CacheTune reduces external-cache I/O while preserving positional consistency in non-prefix KV reuse.
- We model online recomputation and external cache transfer as two overlappable critical paths, and combine hardware profiling with empirical calibration search to adaptively determine the optimal recomputation ratio, enabling CacheTune to sustain low TTFT in multi-tier caching environments.
- We evaluate CacheTune across multiple models and long-context tasks. CacheTune reduces TTFT by 3.72×–4.86×, improves throughput by 3.93×–6.21×, maintains near-full-recompute quality, outperforms state-of-the-art quality–latency trade-offs, and still achieves 2.34×–2.36× TTFT speedup on I/O-bound SSD/HDD cache pools.

2 Related Work

Early work reused only the KV Cache of common prefixes across requests to avoid redundant prefill of high-frequency segments such as prompt templates. Representative systems include vLLM’s PagedAttention [15], SGLang’s RadixAttention [40], and ChunkAttention [39]. The fundamental limitation of these approaches is that they handle only strict prefixes and cannot meet the reuse demand of dynamically concatenated independent chunks in scenarios such as RAG and multi-document summarization.

Consequently, an increasing body of work over the past two years has turned to non-prefix or modular KV Cache reuse. PromptCache [6] is the first to treat the KV Cache as an independently cacheable and composable module, but its reuse scope remains constrained by a predefined prompt schema. CacheBlend [37] is a representative system that addresses the cross-attention loss arising from multi-chunk KV concatenation: it locates the high-KV-deviation (HKVD) token set through a full first-layer recomputation and then

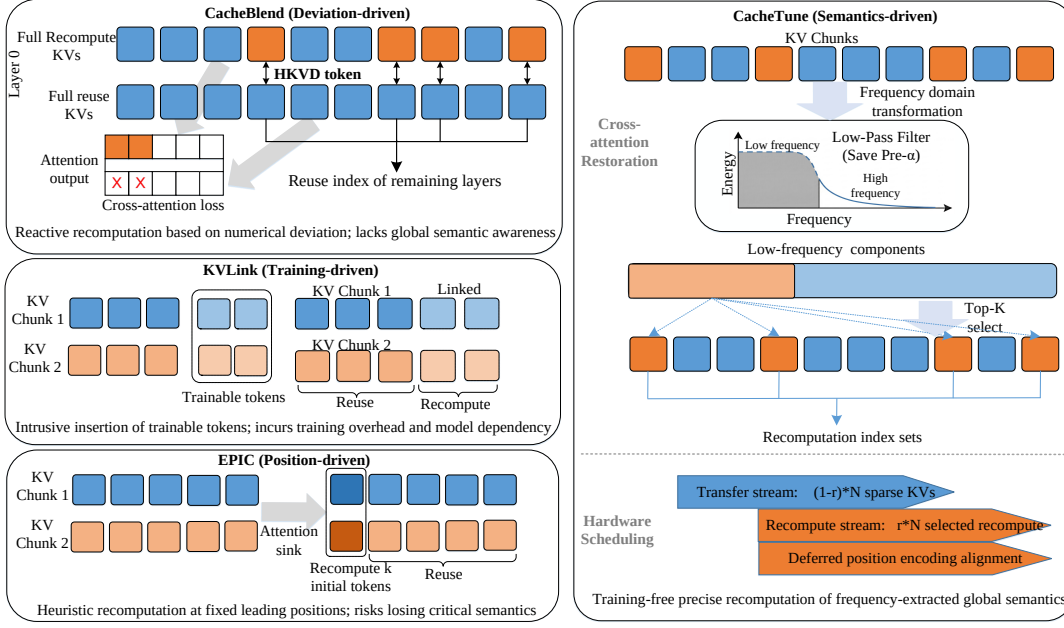


Figure 1. Comparison of cross-attention restoration strategies in non-prefix KV Cache reuse.

selectively recomputes a fixed fraction; however, the first-layer recomputation overhead is irreducible, and its selection is based solely on the first-layer attention deviation. Subsequently, EPIC [10] recomputes only the first k attention-sink [31] positions of each chunk to recover cross-attention at extremely low cost, but this approach offers insufficient coverage of globally semantic tokens and fails to guarantee model accuracy in many scenarios. CacheClip [34] exploits the similarity between the last-layer attention distributions of a small auxiliary LLM and the main model to identify tokens requiring recomputation, with token selection therefore depending on the auxiliary model and a fine-tuning procedure. KVLink [35] inserts learnable link tokens at chunk boundaries to bridge independently encoded KV Chunks; this approach requires additional pre-training and is tightly coupled to specific models. CacheSlide [21] studies Relative-Position-Dependent Caching (RPDC) in agent workflows, where reusable segments preserve their relative order while their absolute positions shift. It mitigates positional drift through Chunked Contextual Position Encoding and Weighted Correction Attention, and improves cache management with spill-aware optimizations. None of the above works addresses cross-attention loss by directly characterizing which tokens truly carry the global semantics; rather, they provide approximations based on attention deviation, absolute position, or learned signals. CacheTune is the first to characterize, from a frequency-domain perspective, the semantic concentration across chunks during multi-chunk KV reuse, and to derive a training-free, model-agnostic token-selection method based on this insight.

Frequency-domain analysis has long been widely applied in image processing, signal processing, and compression coding [33]. Classical image-compression methods exploit the energy concentration of signals in the frequency domain. After transforming the spatial-domain image into the frequency domain, low-frequency components are preferentially retained while high-frequency details are compressed or truncated [19, 29]. A recent work, FreqKV [13], is the first to systematically observe that LLM KV Cache likewise exhibits frequency-domain structural characteristics analogous to those of signals and images. It maps KV from the token-sequence domain to the frequency domain and reduces KV storage overhead by retaining low-frequency components while compressing or discarding high-frequency ones. While FreqKV demonstrates exploitable frequency-domain redundancy in KV Cache, its main goal is KV Cache compression. In contrast, CacheTune leverages KV frequency-domain properties to recover global semantic information lost during non-prefix reuse, repurposing frequency-domain analysis for semantic-importance modeling.

3 Motivation

In a Transformer, the KV of a token at layer ℓ is determined not solely by that token’s local word embedding, but by its hidden state, which has aggregated preceding context through causal self-attention. Let the input to layer ℓ be $H^\ell = [h_1^\ell, h_2^\ell, \dots, h_n^\ell]$; the Key and Value of token i are

$$k_i^\ell = h_i^\ell W_K^\ell, \quad v_i^\ell = h_i^\ell W_V^\ell. \quad (1)$$

Here, h_i^ℓ itself originates from the previous layer’s aggregation over historical tokens. When a chunk is encoded in

isolation, its tokens attend only within the chunk; when placed in a full prompt, they should also attend to preceding chunks. The reused KV therefore deviates from its full-prompt counterpart by missing these inter-chunk dependencies. This missing dependency is the fundamental cause of cross-attention loss.

The KV Cache is essentially a time-domain sequence. To investigate its intrinsic representational properties, we apply the Fast Fourier Transform (FFT) [30] to convert it from the time domain to the frequency domain and find that its energy is strongly concentrated in the low-frequency band. Figure 2 reports the energy distribution of high- and low-frequency components of the KV Cache during inference of Mistral v0.3-7B [12] and Llama 3-8B [27] on the MuSiQue [28] and SAMSum [7] datasets. As clearly observed, the energy proportion of the high-frequency components of both Key and Value decreases monotonically as the frequency increases, while the lowest 20% of the spectrum (the low-frequency band) accounts for the vast majority of the total energy.

To further investigate how KV pairs of different frequency-domain characteristics influence generation quality in non-prefix KV Cache reuse, we feed the model multiple distinct KV Chunks followed by an appended prompt (the suffix query), so that the suffix query naturally induces cross-chunk attention against all historically reused KV Chunks. We then run inference under different recomputation strategies and extract the resulting cross-attention weight matrices in the generation stage. As shown in Figure 3(a), under the full-recompute baseline, the query’s attention over historical context exhibits strong sparsity and local concentration: the model concentrates the vast majority of attention weight on a small number of key KV pairs. Hence, identifying and recomputing only these key KV pairs is sufficient to effectively recover the global attention pattern. Figure 3(c) and Figure 3(d) further show the resulting attention distributions when only the top 15% lowest-frequency components and the top 15% highest-frequency components are selected for recomputation, respectively. We observe that low-frequency-guided recomputation almost completely reconstructs an attention backbone highly consistent with the baseline, whereas high-frequency-guided recomputation remains similar to full reuse (Figure 3(b)) and both introduce substantial deviations in the global attention distribution. Combined with Figure 2, these results suggest that the low-frequency components of KV Cache encode the global contextual information most affected by non-prefix reuse, whereas high-frequency components are less effective for restoring cross-chunk dependencies. Hence, selectively recomputing low-frequency-dominant KV pairs effectively supplies the model with a near-lossless global context summary, enabling cross-chunk attention to be restored during non-prefix reuse generation.

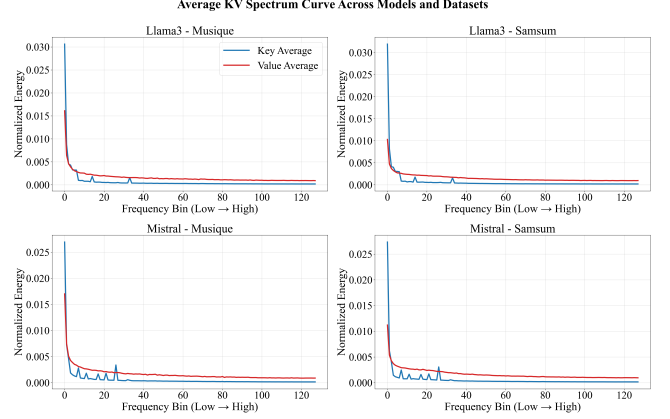


Figure 2. Energy distribution of the KV Cache along the sequence dimension in the frequency domain.

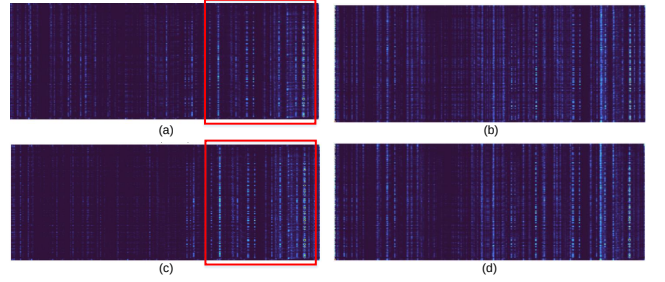


Figure 3. Cross-attention weight heatmaps from the suffix query to historical KV Chunks under different recomputation strategies: (a) full recompute; (b) direct KV Cache reuse without recomputation; (c) recomputing only the tokens corresponding to the top 15% low-frequency components; (d) recomputing only the top 15% high-frequency components.

Beyond the semantic-level cross-attention loss, the practical gain of KV Cache reuse is further constrained by cache-organization granularity. In real systems, reusable KV Caches are typically organized into fixed-length chunks that serve as the basic unit of storage and transfer; because such chunks often occupy substantial space, they are hierarchically off-loaded to larger-capacity media such as CPU memory, SSD/HDD, or even lower-tier external storage. Under this setting, if full recomputation is adopted, larger chunks incur higher forward-computation overhead; if full reuse is adopted, the cost is jointly constrained by the number of chunks, the storage tier, and the transfer granularity. Figure 4 shows the latency trade-off under different chunk sizes and recomputation ratios using the Mistral v0.3-7B model. For CPU-memory cache, a small recomputation ratio is sufficient to maintain low latency because data movement is relatively cheap. In contrast, HDD-based reuse benefits from a larger recomputation ratio, since direct reuse can make I/O the dominant cost. These results indicate that the benefit of KV Cache reuse is

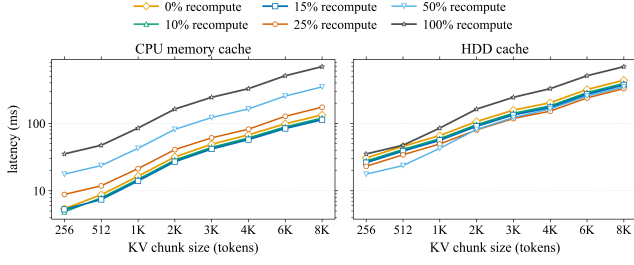


Figure 4. Impact of recomputation ratio on KV Cache reuse latency. The 0% and 100% curves correspond to full KV reuse and full GPU recomputation. CPU-memory cache favors low recomputation ratios, while HDD cache requires more recomputation to reduce I/O overhead.

not determined simply by whether reuse is performed, but by the interaction between the cache medium and recomputation cost. Without unified modeling and coordination of these factors, either I/O or computation may become the critical bottleneck limiting TTFT. Therefore, adaptively balancing cache reuse and recomputation under a given storage hierarchy is a key challenge in building efficient KV Cache reuse systems.

4 Methodology

CacheTune is designed to address the two challenges raised in Section 3: (1) identifying, among all reusable KV Chunks, the key KV pairs whose recomputation can restore the global cross-attention; and (2) minimizing data-transfer overhead via pipeline-level optimizations and adaptively selecting, under a given chunk granularity and storage tier, the recomputation-reuse ratio that drives down end-to-end TTFT. An overview of the CacheTune workflow is illustrated in Figure 5. CacheTune operates in two stages. In the offline stage, frequency-domain analysis is applied to each reusable KV Chunk to precompute, for every layer, the index set of KV pairs most worth recomputing. In the online stage, this precomputed index set is used to select the KV pairs for recomputation, while the remaining KVs are sparsely transferred from the cache pool to the GPU and fused with the recomputation results. To further enhance inference performance, we incorporate cross-layer asynchronous prefetching, a deferred Rotary Position Embedding (RoPE) recovery mechanism, and a hardware-aware adaptive recomputation-ratio scheduler, thereby unifying cross-attention recovery, data movement, and positional consistency within a single inference pipeline.

4.1 Frequency-Domain Selective Recomputation

CacheTune identifies key tokens by performing a frequency-domain decomposition of the KV Cache along the sequence dimension. Let the Key and Value tensors at layer ℓ be $\mathbf{K}^{(\ell)} \in \mathbb{R}^{N \times H \times D}$ and $\mathbf{V}^{(\ell)} \in \mathbb{R}^{N \times H \times D}$, respectively, where N denotes the sequence length, H is the number of KV heads,

and D is the per-head dimension. To analyze the frequency-domain structure of the KV Caches across layers, we adopt the FFT as our spectral analysis tool [30]. As a classical signal-processing primitive, the Fourier transform decomposes a time-domain signal into a set of orthogonal frequency components, thereby explicitly exposing the energy distribution and scale structure of the data within a low-dimensional frequency representation [8, 33]. Moreover, the computational complexity of FFT is $O(N \log N)$, substantially lower than the $O(N^2)$ cost of attention. Considering further that the KV Cache is real-valued, we adopt the real-valued FFT (rFFT), which computes only the non-redundant spectral components and reduces the number of output frequencies from N to $\lfloor N/2 \rfloor + 1$. We apply rFFT along the sequence dimension to both Key and Value:

$$\begin{aligned}\widehat{\mathbf{K}}^{(\ell)} &= \text{rfft}(\mathbf{K}^{(\ell)}, \text{dim} = 0), \\ \widehat{\mathbf{V}}^{(\ell)} &= \text{rfft}(\mathbf{V}^{(\ell)}, \text{dim} = 0),\end{aligned}\tag{2}$$

where $\widehat{\mathbf{K}}^{(\ell)}, \widehat{\mathbf{V}}^{(\ell)} \in \mathbb{C}^{(\lfloor N/2 \rfloor + 1) \times H \times D}$.

We then apply a low-pass filter that retains only the lowest α fraction of frequency components. Letting the cutoff index be $c = \lfloor \alpha \cdot (\lfloor N/2 \rfloor + 1) \rfloor$, the filtered spectrum is given by

$$\begin{aligned}\widehat{\mathbf{K}}_{\text{LP}}^{(\ell)}[k, :, :] &= \begin{cases} \widehat{\mathbf{K}}^{(\ell)}[k, :, :], & k < c, \\ \mathbf{0}, & k \geq c, \end{cases} \\ \widehat{\mathbf{V}}_{\text{LP}}^{(\ell)}[k, :, :] &= \begin{cases} \widehat{\mathbf{V}}^{(\ell)}[k, :, :], & k < c, \\ \mathbf{0}, & k \geq c. \end{cases}\end{aligned}\tag{3}$$

An inverse rFFT then yields the low-frequency reconstruction:

$$\begin{aligned}\widetilde{\mathbf{K}}^{(\ell)} &= \text{irfft}(\widehat{\mathbf{K}}_{\text{LP}}^{(\ell)}, n = N, \text{dim} = 0), \\ \widetilde{\mathbf{V}}^{(\ell)} &= \text{irfft}(\widehat{\mathbf{V}}_{\text{LP}}^{(\ell)}, n = N, \text{dim} = 0).\end{aligned}\tag{4}$$

For the KV pair associated with token i , we define its low-frequency energy score in the Key and Value spaces as

$$s_{i,K}^{(\ell)} = \|\widetilde{\mathbf{K}}_i^{(\ell)}\|_2, \quad s_{i,V}^{(\ell)} = \|\widetilde{\mathbf{V}}_i^{(\ell)}\|_2,\tag{5}$$

respectively. On this basis, the importance of each KV pair is defined as the joint statistic:

$$s_i^{(\ell)} = \frac{1}{2}(s_{i,K}^{(\ell)} + s_{i,V}^{(\ell)}),\tag{6}$$

which balances the contributions of Key and Value to the importance estimate.

Given a recomputation ratio r , we apply TopK to select the top rN KV pairs as the frequency-guided recomputation candidate set:

$$\mathcal{I}_{\text{freq}}^{(\ell)} = \text{TopK}(\{s_i^{(\ell)}\}_{i=1}^N, rN),\tag{7}$$

while its complement $\overline{\mathcal{I}_{\text{freq}}^{(\ell)}}$ corresponds to the KV pairs that can be directly reused and streamed in from the cache pool. In this manner, CacheTune identifies semantically critical

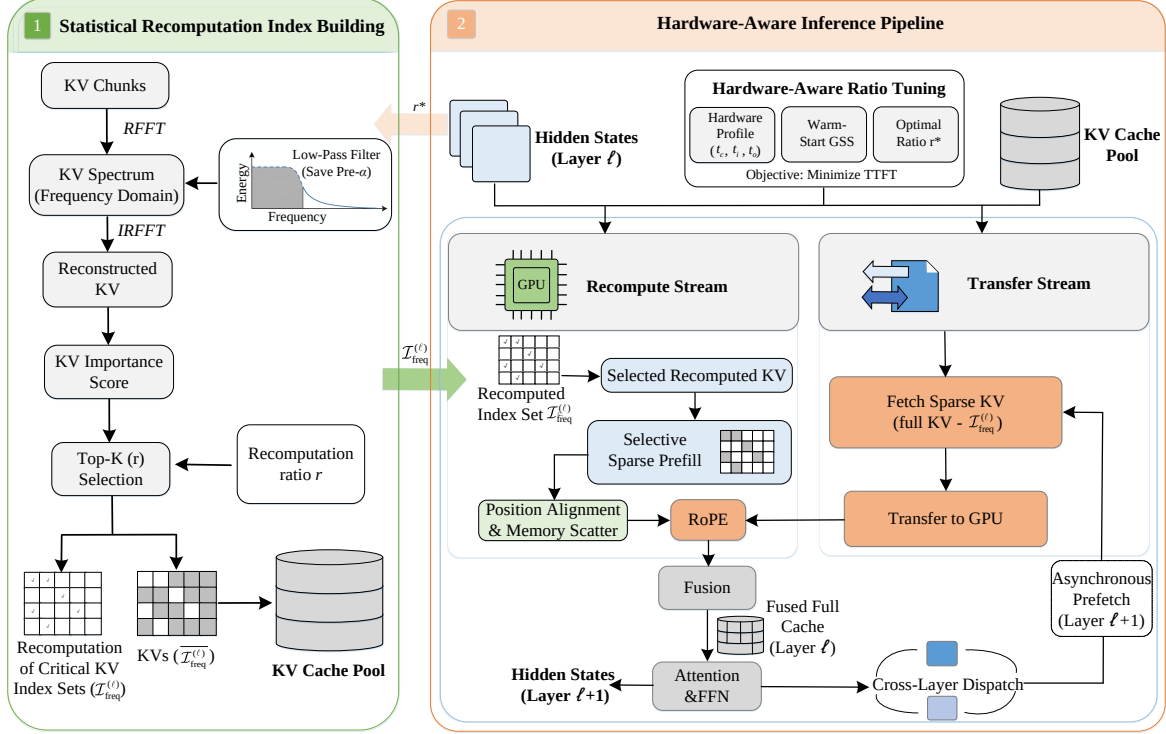


Figure 5. Overview of CacheTune.

tokens during offline KV Cache generation and, at online inference time, fuses their selectively recomputed KVs with the directly reused KVs of the remaining tokens.

4.2 KV Cache Offloading and Sparse Reuse

To reduce GPU memory pressure, CacheTune places pre-computed KV Chunks in a GPU-external cache pool, which may reside in CPU memory, SSDs, or lower-tier storage. A key design of CacheTune is that the frequency-domain re-computation indices are not used only for semantic recovery, but are also exposed to the cache system as an I/O plan. Specifically, for each layer ℓ , the offline frequency analysis produces a recombination set $\mathcal{I}_{\text{req}}^{(\ell)}$. During online inference, CacheTune does not fetch a full KV Chunk from the external cache pool. Instead, it transfers only the complementary KV pairs, $\mathcal{I}_{\text{req}}^{(\ell)}$, and leaves the selected semantic-critical tokens to be regenerated by online re-computation. In this way, the same frequency-guided index set simultaneously determines which KV pairs to recompute and which to skip during cache loading, reducing the transferred KV volume from N to $(1-r)N$ while preserving the tokens that are most important for cross-attention recovery.

Figure 6 illustrates CacheTune’s sparse KV reuse pipeline. The main idea is to convert non-prefix KV reuse from a full-cache loading problem into an index-aware fusion problem: guided by the recombination index set, CacheTune

fetches only the reusable non-selected KVs from the external cache pool, recomputes the semantically critical KVs under the true global context, and scatters the two parts back into a complete layer-wise KV Cache. This design jointly addresses three coupled challenges in non-prefix reuse: external-cache transfer overhead, online re-computation overhead, and positional-encoding inconsistency.

Hiding external-cache transfer overhead. To hide data movement behind computation, CacheTune introduces two dedicated CUDA streams in addition to the model’s native forward compute stream. The Transfer stream asynchronously prefetches the non-recomputed KV pairs required by the next layer from the cache pool to the GPU, while the Recompute stream regenerates the selected tokens in $\mathcal{I}_{\text{req}}^{(\ell)}$ and prepares position-corrected Keys. The streams are coordinated by CUDA Events at layer granularity. While the forward stream executes the current layer, the Transfer stream preloads the reusable KVs of the next layer, and the Recompute stream prepares the selected KVs. Therefore, when the next layer starts, its cache-loading latency has largely been hidden behind the previous layer’s computation.

Reducing online re-computation and fusion overhead. To efficiently assemble the complete layer-wise KV Cache, CacheTune maintains reusable per-layer working buffers and performs scatter fusion between transferred and recomputed KVs. During online prefilling, the system categorizes active layers into three classes: (1) the collection layer,

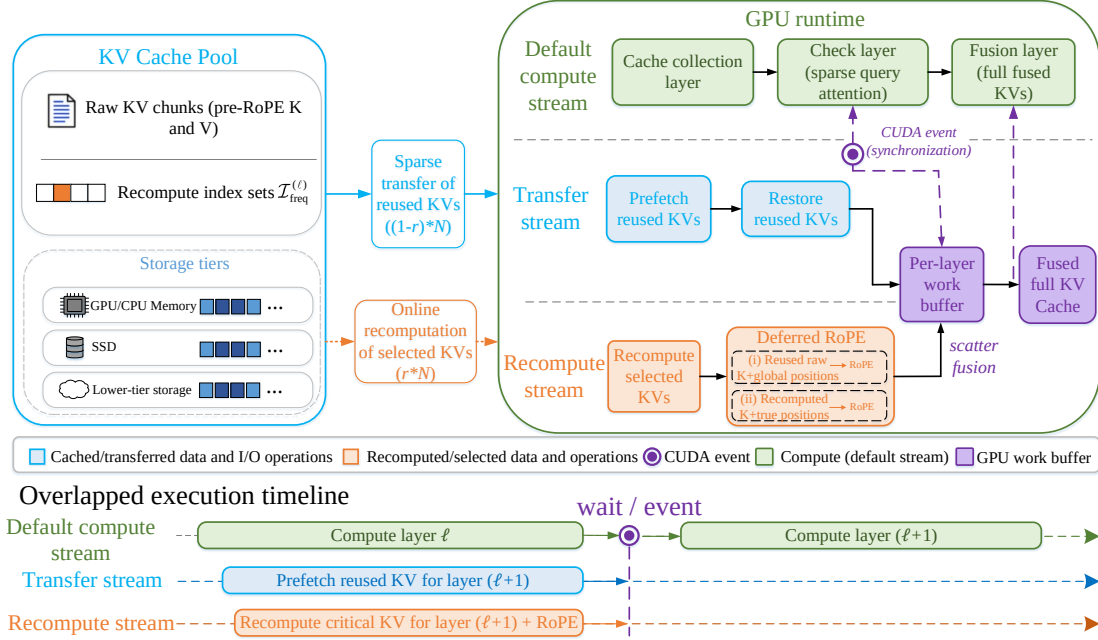


Figure 6. KV Cache offloading and sparse reuse pipeline.

which gathers the offline KV Chunks intended for reuse; (2) the check layer, where the system applies the precomputed index set to sparsely filter the query and computes attention only for the corresponding tokens; and (3) the fusion layers, comprising every layer subsequent to the check layer. For each Fusion layer, the system waits for the sparse transfer to complete, restores the reused KVs into their original positional indices, and scatters the recomputed KVs into their corresponding positions, thereby assembling the complete KV Cache within a single buffer.

Preserving positional consistency. For RoPE-based LLMs, positional information is injected into Keys after the linear projection. If each chunk is cached after applying RoPE with local positions, reassembling multiple chunks into a new global prompt will cause positional encodings to deviate from their true global positions. To resolve this, CacheTune introduces a deferred RoPE recovery mechanism. Instead of caching post-RoPE Keys during offline KV generation, CacheTune stores raw pre-RoPE Keys in the cache pool. During online inference, after the reused Keys are prefetched to the GPU by the Transfer stream, the Recompute stream applies RoPE using their true global positions p_i^{global} in the full sequence:

$$\mathbf{K}_{\text{reuse},i}^{(\ell)} = \text{RoPE}\left(\mathbf{K}_{\text{raw},i}^{(\ell)}, p_i^{\text{global}}\right), \quad i \in \overline{\mathcal{I}_{\text{freq}}^{(\ell)}} \quad (8)$$

For the Keys produced by current-layer recomputation, standard RoPE is applied directly with the true positions of the current layer. Both the reused and the recomputed Keys are thereby mapped into a single global coordinate frame and, via scatter fusion within the working buffer, assembled into

the complete KV. This design not only fundamentally eliminates the positional-encoding misalignment of non-prefix reuse but, since RoPE is folded into the asynchronous execution path of the Recompute stream, incurs no additional serial latency—positional recovery is overlapped with the sparse-transfer process.

4.3 Hardware-Aware Co-Scheduling of Recomputation and Transfer

When the reusable KV Cache resides on a slow external storage medium, the principal tension of a KV-reuse system is no longer merely whether redundant computation is avoided; it evolves into an end-to-end latency optimization problem jointly governed by storage-tier bandwidth, access granularity, asynchronous DMA efficiency, online recomputation overhead, and the inter-layer pipeline organization. The optimal recomputation ratio r varies substantially across different hardware combinations, and any scheme that fixes r across all environments fails to remain optimal under heterogeneous deployment. We therefore cast the choice of r as a hardware-aware pipeline optimization problem: given the transfer and compute characteristics of the target hardware, find the r^* that minimizes TTFT. Let N denote the context length at a given layer; the number of KV pairs requiring online recomputation is rN , and the number of tokens to be loaded from the cache pool is $(1-r)N$. We further let t_c denote the per-token single-layer recomputation cost, governed by GPU compute capability and model width, and let t_i denote the per-token single-layer effective transfer cost, governed by the cache pool’s read/write bandwidth,

DMA granularity, and PCIe utilization. Finally, t_o captures the per-layer fixed pipeline overhead, encompassing kernel launches, CUDA-Event synchronization, allocator invocations, and other terms independent of r . The quantities t_c , t_i , and t_o can all be measured by a one-time hardware-profiling step during deployment initialization.

Under the multi-stream pipeline, layer $\ell+1$ cache transfer overlaps with layer ℓ recomputation and forward execution, so the per-layer latency is bounded by the slower path:

$$T^{(\ell)}(r) = \max(rN \cdot t_c, (1-r)N \cdot t_i) + t_o. \quad (9)$$

For the ℓ -layer pipeline, after neglecting the constant bubble terms at start-up and tear-down, the steady-state total latency is approximately

$$T_{\text{TTFT}}(r) \approx \ell \cdot \max(rNt_c, (1-r)Nt_i) + \ell \cdot t_o. \quad (10)$$

The TTFT model in Equation 10 exhibits a classical roofline structure: when r is small, the system lies in the I/O-bound regime, and TTFT is dominated by cache transfer; when r is large, the system enters the compute-bound regime, and latency is dominated by GPU recomputation. The crossover point at which the transfer time equals the recomputation time yields the first-order analytical optimum:

$$r_0 = \frac{t_i}{t_c + t_i}. \quad (11)$$

To avoid overly small ratios that harm cross-attention recovery on high-bandwidth media (i.e., GPU or CPU memory), we impose a quality-preserving lower bound $r_{\min} = 15\%$, following the effect of recomputation ratio r in Section 5.2.

The analytical model above holds under idealized pipelining assumptions, yet real systems exhibit a number of second-order effects that resist closed-form characterization, such as nonlinear DMA start-up costs and pipeline bubbles. A more elaborate procedure is therefore required to obtain the true optimal r^* . To this end, CacheTune adopts a two-stage scheduling strategy. It first uses the analytical solution r_0 as a prior anchor, and then refines r^* around r_0 using a small calibration set. The calibration set $\mathcal{S}_{\text{cal}}$ can be sampled from the target workload or a held-out benchmark. Since calibration measures TTFT rather than task-specific semantics, only a few samples are required. Let the calibration set be $\mathcal{S}_{\text{cal}} = \{s_1, s_2, \dots, s_{|\mathcal{S}_{\text{cal}}|}\}$ and define the mean TTFT as the objective function:

$$f(r) = \frac{1}{|\mathcal{S}_{\text{cal}}|} \sum_{s \in \mathcal{S}_{\text{cal}}} \text{TTFT}(s; r), \quad (12)$$

with the optimization target

$$r^* = \arg \min_{r \in [r_{\min}, r_{\max}]} f(r). \quad (13)$$

From the preceding roofline analysis, $f(r)$ is approximately unimodal over its global domain (decreasing on the low- r side dominated by I/O, and increasing on the high- r side dominated by computation). We therefore adopt golden-section search (GSS) [14] to solve for r^* . GSS requires no

Algorithm 1 Roofline-Warmstart Golden Section Search for Optimal Recomputation Ratio

Require: Calibration set $\mathcal{S}_{\text{cal}}$; model $\mathcal{M}$; roofline prior $r_0 = t_i/(t_c + t_i)$, derived from hardware profile (t_c, t_i) and clipped to $[r_{\min}, r_{\max}]$; semantic bounds $r_{\min}, r_{\max}$ with $r_{\min} \leq r_{\max}$; tolerance threshold ε .

Ensure: Optimal recomputation ratio r^* .

```

1:  $a \leftarrow r_{\min}; b \leftarrow r_{\max}$  ▷ Initialize search bounds
2:  $\phi \leftarrow (\sqrt{5} - 1)/2$  ▷ Golden ratio  $\approx 0.618$ 
3: if  $r_0 \leq (a + b)/2$  then
4:    $x_1 \leftarrow r_0; x_2 \leftarrow a + \phi \cdot (b - a)$  ▷  $r_0$  in left half;
   warm-start left probe
5: else
6:    $x_1 \leftarrow b - \phi \cdot (b - a); x_2 \leftarrow r_0$  ▷  $r_0$  in right half;
   warm-start right probe
7: end if
8:  $f_1 \leftarrow \text{EVALTTFT}(\mathcal{M}, \mathcal{S}_{\text{cal}}, x_1)$  ▷ Mean TTFT over
   calibration set
9:  $f_2 \leftarrow \text{EVALTTFT}(\mathcal{M}, \mathcal{S}_{\text{cal}}, x_2)$ 
10: while  $|b - a| \geq \varepsilon$  do
11:   if  $f_1 < f_2$  then
12:      $b \leftarrow x_2; x_2 \leftarrow x_1; f_2 \leftarrow f_1$  ▷ Optimum in  $[a, x_2]$ ;
     reuse  $f_1$ 
13:      $x_1 \leftarrow b - \phi \cdot (b - a)$ 
14:      $f_1 \leftarrow \text{EVALTTFT}(\mathcal{M}, \mathcal{S}_{\text{cal}}, x_1)$  ▷ One new
     evaluation per iteration
15:   else
16:      $a \leftarrow x_1; x_1 \leftarrow x_2; f_1 \leftarrow f_2$  ▷ Optimum in  $[x_1, b]$ ;
     reuse  $f_2$ 
17:      $x_2 \leftarrow a + \phi \cdot (b - a)$ 
18:      $f_2 \leftarrow \text{EVALTTFT}(\mathcal{M}, \mathcal{S}_{\text{cal}}, x_2)$ 
19:   end if
20: end while
21:  $r^* \leftarrow (a + b)/2$ 
22: return  $r^*$ 

```

gradient information, performs one function evaluation per iteration, and converges within a tolerance ε in only about $\lceil \log_{1/\phi}(1/\varepsilon) \rceil$ evaluations. The overall calibration cost is thus $\mathcal{O}(|\mathcal{S}_{\text{cal}}| \cdot \log(1/\varepsilon))$ TTFT measurements. Compared with naive grid search that treats $f(r)$ as a black box, our scheme uses the analytical estimate r_0 from hardware profiling to warm-start GSS, placing one initial probe near the theoretical optimum and thus accelerating convergence. The algorithm flow is presented in Algorithm 1. The profiled quantities t_c , t_i , and t_o are determined by the model architecture and the target hardware/storage path, rather than by the particular downstream workload. Therefore, a single profiling-and-calibration pass at deployment time yields the r^* .

5 Experimental Evaluation

The experimental evaluation is organized around three research questions: Q1. Can CacheTune achieve a better accuracy–TTFT trade-off than existing methods in non-prefix KV Cache reuse scenarios? Q2. Can frequency-domain selection identify the KV pairs critical to global semantic recovery? Q3. Across different storage tiers, can CacheTune’s hardware-aware scheduler adaptively sustain its TTFT advantage?

5.1 Experimental Setup

Models. We use three open-source LLMs of different scales, including Mistral v0.3-7B [12], Llama 3-8B [27], and Qwen 2.5-32B [11], to validate the effectiveness of CacheTune.

Datasets and metrics. We evaluate on five long-context task datasets; their characteristics and evaluation metrics are summarized in Table 1.

Hardware platforms. We evaluate CacheTune on two platforms: (i) the A100 platform with 2× NVIDIA A100-40GB GPUs connected via PCIe Gen3×16, and (ii) the RTX 4090 platform with 2× NVIDIA GeForce RTX 4090-24GB GPUs, also connected to the host via PCIe Gen3×16. Disk-offload media: the A100 platform uses an HDD-backed cache pool, with fio sequential read/write yielding effective read and write bandwidths of the HDD-backed offload path of approximately 205 MB/s and 201 MB/s. The RTX 4090 platform uses an SSD-backed cache pool, with fio measuring effective read and write bandwidths of the SSD-backed offload path of approximately 535 MB/s and 445 MB/s. We use these two storage media to evaluate CacheTune’s hardware-aware recomputation capability under disparate disk bandwidths.

Baselines. We compare CacheTune with seven methods:

1. **Full Recompute:** standard prefill over the complete prompt; this serves as the most basic **baseline**.
2. **Full KV Reuse:** directly concatenates independently encoded KV Chunks without any recomputation.
3. **vLLM Prefix Caching [15]:** uses vLLM’s native Automatic Prefix Caching, which can only reuse strict prefixes; non-prefix chunks that cannot be matched as a strict prefix are recomputed via the default vLLM prefill path.
4. **CacheBlend [37]:** performs full first-layer recomputation to obtain the HKVD tokens; subsequent layers selectively recompute the same subset, with the recomputation ratio set to 15%.
5. **EPIC [10]:** recomputes only the attention-sink positions (the first 16 tokens in our experiments) to recover global semantics.
6. **CacheSlide [21]:** We adapted it to our multi-chunk setting by treating reusable document chunks as fixed segments and the query as the updated segment, and used its Weighted Correction Attention to selectively recompute and fuse KV pairs from reused chunks.

7. **KVLink [35]:** pretrains learnable link tokens and inserts them at chunk boundaries; in our experiments, 5 additional link tokens were inserted per chunk.

Default hyperparameters. We set the low-frequency cutoff ratio to $\alpha = 0.5$. Since the resulting L2-norm scores are consumed by a TopK selector that depends only on the relative ranking of tokens rather than on absolute score magnitudes, the method is largely insensitive to the precise cutoff. We empirically verified that the resulting TopK selection remains stable for any $\alpha \in [0.3, 0.7]$, and adopt $\alpha = 0.5$ as a representative default that captures the dominant low-frequency band.

All experiments were conducted on Ubuntu 22.04.1 with Linux kernel 6.8.0, using PyTorch 2.2.1 and CUDA 12.4.

5.2 Performance Evaluation

Accuracy–TTFT trade-off. We compare CacheTune with baseline methods in terms of accuracy and TTFT. Since methods such as EPIC and KVLink do not perform multi-tier KV Cache storage optimization, in the following results, these methods do not offload the KV Cache; all reusable KV Chunks reside in the fastest-access GPU memory, which represents their best-case TTFT. CacheTune offloads reusable KV Chunks to CPU memory and applies the optimized offload pipeline in Section 4.2. Since CPU memory is a high-speed cache medium, the hardware-aware recomputation selects the quality-preserving lower bound $r = r_{\min} = 15\%$ for CacheTune in this setting.

Figure 7 reports the overall trade-off. Full Recompute provides the accuracy upper bound but incurs the highest TTFT, while Full KV Reuse achieves the lowest TTFT at the cost of severe accuracy degradation caused by missing cross-chunk attention. vLLM Prefix Caching preserves the accuracy of Full Recompute by losslessly reusing matched prefix KV pairs, but it cannot exploit reusable non-prefix chunks that are dynamically composed in workloads. As a result, many reusable chunks still need to be recomputed, leading to higher TTFT than CacheTune. CacheTune achieves accuracy close to Full Recompute, retaining on average 94.8% of the generation quality across five datasets and three models, while reducing TTFT by 3.72×–4.86×. Compared with CacheBlend under the same 15% recomputation ratio, CacheTune achieves both higher accuracy and lower TTFT. It also outperforms CacheSlide in the overall accuracy–TTFT trade-off. CacheSlide mainly addresses RPDC-style agent prompts, where reusable segments preserve their relative order, and positional drift is the dominant issue. In contrast, CacheTune targets independently precomputed chunks in RAG, multi-document QA, and summarization, where recovering cross-chunk global semantics is more important. This jointly demonstrates that CacheTune’s frequency-domain selection recovers cross-attention more effectively, while its sparse offloading and prefetching pipeline hides most

Table 1. Summary of evaluation datasets and metrics.

Dataset	Task type	Context construction	Metric $\uparrow$
SAMSum [7]	Multi-turn dialogue summarization	Concatenation of multi-turn dialogues	ROUGE-L [18]
MuSiQue [28]	Multi-hop QA	Concatenation of multi-document retrieval chunks	F1
WikiMQA [9]	Multi-hop QA	Concatenation of multi-document retrieval chunks	F1
HotpotQA [36]	Multi-hop QA	Concatenation of multi-document retrieval chunks	F1
Multi-News [4]	Multi-document summarization	Concatenation of multiple news articles	ROUGE-L

transfer overhead. Unlike CacheBlend and CacheSlide, which require online first-layer recomputation to identify recomputation targets, CacheTune derives recomputation indices offline when KV Chunks are generated. Therefore, it can discard the recomputation-targeted KV pairs during offloading, transfer only the complementary KVs, and avoid extra HKVD-style token-selection overhead, yielding lower TTFT. As for ultra-fast systems such as EPIC and KVLink, although they achieve lower TTFT, they suffer substantial accuracy degradation. EPIC recomputes only a small number of tokens at the beginning of each chunk; this fixed-position prior fails to cover the semantically critical tokens distributed in the middle or tail of a chunk, leading to non-trivial accuracy losses across multiple datasets. KVLink requires model pretraining to obtain the link tokens, incurring substantial offline computation cost. By contrast, CacheTune’s frequency-domain analysis is extremely lightweight: for a 3K-token KV Chunk, generating the recomputation index set takes only ~ 0.69 ms on average on the GPU, and the resulting indices can be reused across inference tasks.

Serving throughput under increasing request rates.

As shown in Figure 8, we compare the TTFT of different methods under varying request rates across several datasets and models. Full Recompute reaches saturation even at relatively low request rates, causing TTFT to increase rapidly. CacheBlend and CacheSlide improve throughput, but their latency still grows significantly under high load. In contrast, CacheTune pushes the saturation point to higher request rates while maintaining lower TTFT, thereby achieving higher throughput. Compared with Full Recompute, CacheTune improves throughput by $3.93\times$ – $6.21\times$, compared with $2.71\times$ – $5.16\times$ for CacheBlend and $3.79\times$ – $5.94\times$ for CacheSlide, demonstrating that CacheTune not only reduces the prefill latency of individual requests but also improves the effective serving capacity under different load levels.

Effect of recomputation ratio r . Using the Mistral model, we evaluate how model accuracy and TTFT vary with r ; the results are shown in Figure 9. As r increases from 5% to 25%, model quality improves steadily but with diminishing returns, while the TTFT speedup over Full Recompute decreases markedly. This reveals a clear quality–latency trade-off in selective recomputation. In particular, $r = 15\%$ is sufficient to recover most cross-attention loss while still sustaining a TTFT speedup of over $4\times$. Therefore, we adopt

$r = 15\%$ as the quality-constrained default in our main experiments. When the KV Cache resides in GPU or CPU memory, the low access and transfer costs cause the TTFT-optimal recomputation ratio derived by the hardware-aware model in Section 4.3 to fall below 10%. However, such a low ratio incurs more pronounced quality loss. Thus, for high-speed cache media, we use 15% as the lower-bound recomputation ratio to prioritize accuracy close to Full Recompute. In Section 5.3.2, we further report the adaptively selected recomputation ratios under lower-tier storage media.

5.3 Ablation Studies

To further validate CacheTune’s key mechanisms, we conduct ablation studies to answer two questions: (1) Can the frequency-domain analysis accurately locate the key KV pairs requiring recomputation? (2) Can multi-tier cache offloading and hardware-aware recomputation reduce TTFT in external-cache-pool scenarios?

5.3.1 Effectiveness of the Frequency-Selection Mechanism.

To verify whether the frequency-domain importance score correctly identifies the KV pairs critical to global semantic recovery, we compare three selection schemes at the same recomputation ratio of $r = 15\%$ (so that they incur identical online recomputation overhead): random selection, high-frequency-based selection, and CacheTune’s low-frequency-based selection. The results are shown in Figure 10. Random selection mitigates the quality degradation of non-prefix reuse only to a limited extent, and high-frequency selection performs comparably to random; neither is able to adequately restore the cross-chunk global semantic dependencies. In contrast, CacheTune’s low-frequency selection yields a markedly higher generation quality, confirming that low-frequency KV pairs more stably reflect the long-range dependencies and global semantic structure across chunks.

5.3.2 Storage-Tier and Hardware-Aware Recomputation Analysis.

To verify CacheTune’s adaptability under different cache-pool media, we further evaluate the performance variations when the reusable KV Cache is stored in GPU memory, CPU memory, SSD, and HDD, respectively.

Effectiveness of the optimized offload pipeline. Before evaluating lower-tier storage devices, we first study whether CPU memory can serve as an external KV Cache pool without making CPU-to-GPU transfer the dominant

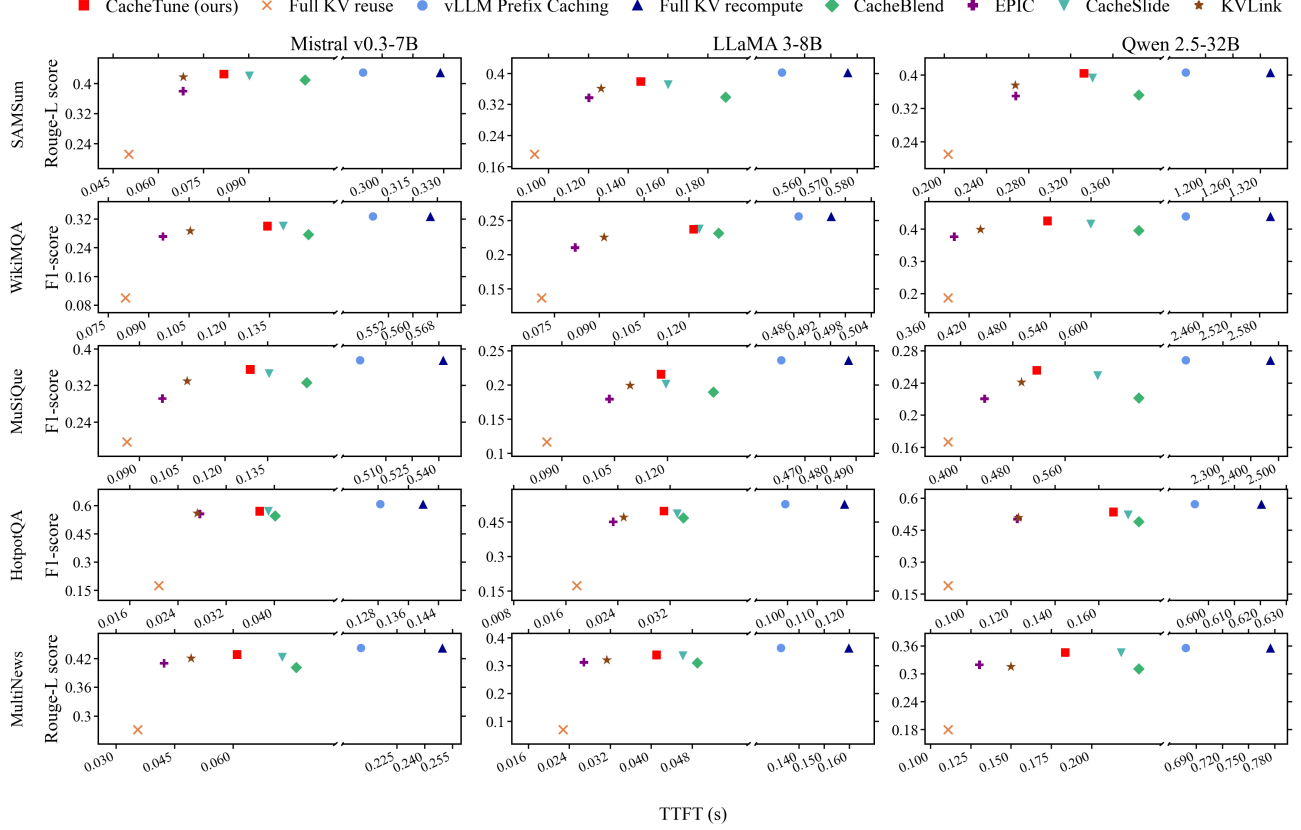


Figure 7. Accuracy–TTFT trade-off of CacheTune compared with baseline methods across different models and dataset tasks.

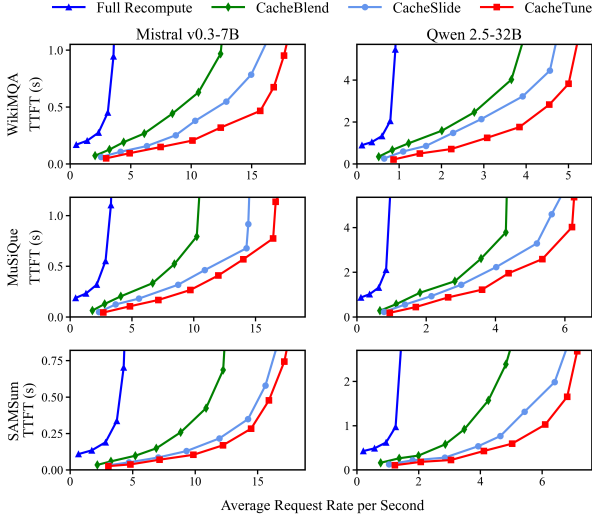


Figure 8. TTFT trends under increasing request rates. Curves extending further to the right with lower TTFT indicate higher effective throughput under low-latency serving.

TTFT bottleneck. We compare CacheTune’s TTFT under a 15% recomputation ratio when the KV Cache is stored in

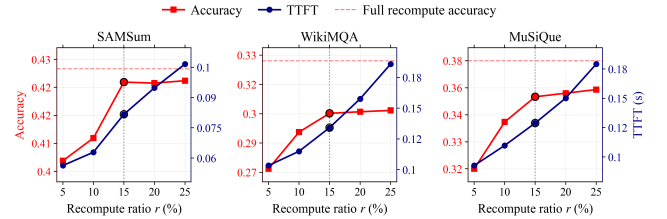


Figure 9. Effect of the recomputation ratio r on model accuracy and TTFT speedup.

GPU memory versus CPU memory. For comparison, we further include CacheBlend, CacheSlide, and vLLM’s native Prefix Caching, all configured with the same cache medium as CacheTune. The results, using Mistral, are reported in Table 2. For GPU-resident caches, CacheTune follows the native GPU cache-reuse path; for CPU-resident caches, it enables the optimized CPU-offload pipeline described in Section 4.2, including sparse reuse, asynchronous transfer, and deferred RoPE recovery. The results show that, even when reusable KVs are placed in CPU memory, these optimizations hide the CPU–GPU transfer and positional-recovery overhead, allowing CacheTune to achieve TTFT comparable to, and even

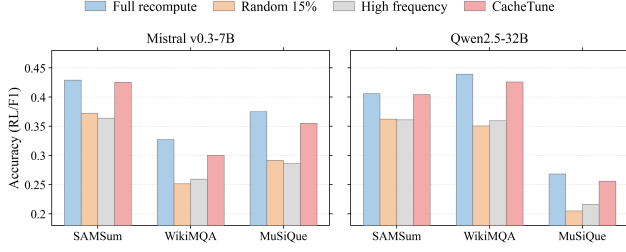


Figure 10. Effectiveness comparison of different recomputation-token selection strategies.

Table 2. TTFT (s) comparison between GPU-resident reuse and the optimized CPU-offload pipeline.

Methods	GPU cache pool			CPU cache pool		
	SAMSsum	WikiMQA	MuSiQue	SAMSsum	WikiMQA	MuSiQue
Full recompute	0.3283	0.5657	0.5426	0.3283	0.5657	0.5426
Prefix Caching	0.2908	0.5469	0.4955	0.3012	0.5496	0.5071
CacheBlend	0.1086	0.1495	0.1487	0.1113	0.1768	0.1723
CacheSlide	0.09064	0.1480	0.1435	0.0995	0.1404	0.1492
CacheTune	0.0899	0.1485	0.1398	0.0817	0.1342	0.1289

lower than, GPU-resident reuse. Compared with CacheBlend, CacheSlide, and vLLM Prefix Caching under the same CPU-offloading setting, CacheTune reduces the average TTFT by 25.3%, 11.9%, and 74.4%, respectively. This confirms that CacheTune’s sparse transfer and asynchronous prefetching effectively alleviate CPU-to-GPU data movement, preventing cache loading from becoming the dominant bottleneck of end-to-end inference.

SSD/HDD as an external cache pool. We further evaluated KV Cache offloading to HDD and SSD on the two hardware platforms described in Section 5.1. Figure 11 reports the TTFT variations under different platforms after applying our hardware-aware adaptive recomputation, with Mistral as the model. When the external cache medium is HDD or SSD, the prohibitive read/write cost limits CacheTune’s TTFT speedup over Full Recompute to only 1.92× and 2.05× on average under a fixed 15% recomputation ratio. With the hardware-aware adaptive scheme proposed in Section 4.3, CacheTune first uses the analytical model as an initial estimate and then refines the recomputation ratio through a lightweight calibration search. In our experiments, we used the first 10 samples from the SAMSsum dataset as the calibration set. Since calibration measures TTFT rather than task-specific semantic quality, only a few requests are needed; empirically, 5–10 samples are sufficient to obtain a stable r^* in our tested configurations. The entire profiling and search process took about 3–4 minutes for each hardware/storage configuration. We use SAMSsum only to provide representative request lengths for latency calibration; the searched

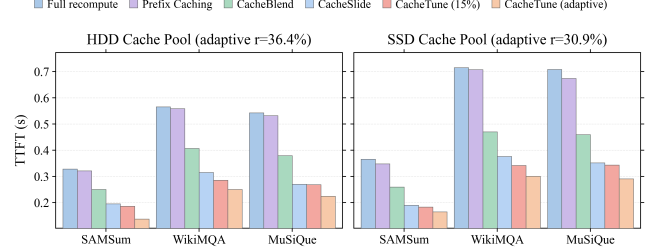


Figure 11. TTFT under different external KV Cache media and recomputation strategies.

ratio is applied per reusable KV Chunk and is determined by the compute-I/O balance of the hardware/storage configuration rather than by the dataset. This cost is incurred only once when deploying CacheTune on a new cache medium or hardware platform, and the resulting recomputation ratio can be reused for subsequent inference requests. The calibration search selects recomputation ratios of 36.4% and 30.9% for HDD and SSD, respectively, increasing the TTFT speedup over Full Recompute to 2.36× and 2.34× on average, while also delivering larger improvements over CacheBlend, CacheSlide, and vLLM Prefix Caching.

These results not only validate the effectiveness of our adaptive recomputation method but also reveal that, when the cache medium is slow, the system should not blindly maximize KV reuse but instead increase the online recomputation ratio appropriately to reduce data read from slow external storage. Moreover, as Figure 9 shows, increasing the recomputation ratio also improves model accuracy, making this an especially favorable trade-off.

5.4 Discussion

Our experiments demonstrate that CacheTune delivers a stable accuracy–latency trade-off in non-prefix KV Cache reuse: the frequency-domain selection mechanism more accurately identifies the KV pairs critical to global semantic recovery, enabling generation quality close to Full Recompute under a low recomputation ratio; sparse transfer, multi-stream overlapping, and deferred RoPE jointly suppress the transfer and positional-recovery overheads of external cache pools; and on slow cache media, the hardware-aware scheduler adaptively raises the recomputation ratio to alleviate the I/O bottleneck, allowing CacheTune to sustain a low TTFT. CacheTune’s cache-pool abstraction further has the potential to extend to emerging heterogeneous memory architectures. For example, with memory-interconnect technologies such as Compute Express Link (CXL) [3, 25], the extended memory beyond the GPU and CPU can be organized as a more unified remote cache pool; CacheTune can still sense the effective transfer cost via hardware profiling and adaptively tune the recomputation–reuse ratio.

CacheTune still has several limitations. When the external storage bandwidth is extremely low or the asynchronous overlap window is insufficient, the system may still be limited by I/O. In future work, we will further deploy CacheTune under higher concurrency, CXL-based disaggregated memory, and more complex workloads.

6 Conclusion

To address the cross-attention loss and the heterogeneous-storage I/O bottleneck in non-prefix KV Cache reuse for long-context LLM inference, this paper presents CacheTune. By leveraging the low-frequency energy concentration of the KV Cache to identify globally semantic key tokens, and by combining sparse transfer, deferred RoPE, multi-stream pipelining, and hardware-aware adaptive recomputation, CacheTune achieves a strong accuracy-latency trade-off across GPU, CPU, SSD, and HDD cache tiers. Across multiple mainstream models and tasks, CacheTune maintains near-full-recompute accuracy while achieving $3.72\times\text{--}4.86\times$ TTFT speedup and $3.93\times\text{--}6.21\times$ higher throughput.

References

- [1] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.
- [2] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. 2024. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*. 3119–3137.
- [3] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Comput. Surv.* 56, 11, Article 290 (July 2024), 37 pages. doi:10.1145/3669900
- [4] Alexander Richard Fabbri, Irene Li, Tianwei She, Suyi Li, and Dragomir Radev. 2019. Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model. In *Proceedings of the 57th annual meeting of the association for computational linguistics*. 1074–1084.
- [5] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX annual technical conference (USENIX ATC 24)*. 111–126.
- [6] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems* 6 (2024), 325–338.
- [7] Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. 2019. SAMSum corpus: A human-annotated dialogue dataset for abstractive summarization. In *Proceedings of the 2nd Workshop on New Frontiers in Summarization*. 70–79.
- [8] Ziwei He, Meng Yang, Minwei Feng, Jingcheng Yin, Xinbing Wang, Jingwen Leng, and Zhouhan Lin. 2023. Fourier transformer: Fast long range modeling by removing sequence redundancy with fft operator. In *Findings of the Association for Computational Linguistics: ACL 2023*. 8954–8966.
- [9] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. 6609–6625.
- [10] Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Qin Zhang, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. 2025. EPIC: Efficient Position-Independent Caching for Serving Large Language Models. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 (Proceedings of Machine Learning Research)*, Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR / OpenReview.net. <https://proceedings.mlr.press/v267/hu25j.html>
- [11] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, An Yang, Rui Men, Fei Huang, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. *CoRR abs/2409.12186* (2024). arXiv:2409.12186 doi:10.48550/ARXIV.2409.12186
- [12] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *CoRR abs/2310.06825* (2023). arXiv:2310.06825 doi:10.48550/ARXIV.2310.06825
- [13] Jushi Kai, Boyi Zeng, Yixuan Wang, Haoli Bai, Ziwei He, Bo Jiang, and Zhouhan Lin. 2025. Freqkv: Frequency domain key-value compression for efficient context window extension. *arXiv preprint arXiv:2505.00570* (2025).
- [14] Mykel J Kochenderfer and Tim A Wheeler. 2019. *Algorithms for optimization*. Mit Press.
- [15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [16] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. {InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [18] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.
- [19] Song Liu, Fei Li, Chenyu Zhao, Qin Xia, Shiqiang Nie, Jinyu Wang, and Weiguo Wu. 2026. FDSR: Efficient Model Training via Adaptive Tensor Quantization Based on Frequency Domain Division and Similarity Data Reuse. *ACM Trans. Archit. Code Optim.* (April 2026). doi:10.1145/3802593 Just Accepted.
- [20] Yuhao Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, et al. 2025. Lmcache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665* (2025).
- [21] Yang Liu, Yunfei Gu, Liqiang Zhang, Chentao Wu, Guangtao Xue, Jie Li, Minyi Guo, Junhao Hu, and Jie Meng. 2026. CacheSlide: Unlocking Cross Position-Aware KV Cache Reuse for Accelerating LLM Serving. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*. 83–99.
- [22] Yuhao Liu, Hanchen Li, Kuntai Du, Jiayi Yao, Yihua Cheng, Yuyang Huang, Shan Lu, Michael Maire, Henry Hoffmann, Ari Holtzman, et al. 2023. Cachegen: Fast context loading for language model applications.

- arXiv preprint arXiv:2310.07240* (2023).
- [23] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2024. Mooncake: A kv-cache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* (2024).
 - [24] Zebin Ren, Krijn Doekemeijer, Tiziano De Matteis, Christian Pinto, Radu Stoica, and Animesh Trivedi. 2025. An i/o characterizing study of offloading llm models and kv caches to nvme ssd. In *Proceedings of the 5th Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*. 23–33.
 - [25] Minseok Seo, Jungi Hyun, Seongho Jeong, Xuan Truong Nguyen, Hyuk-Jae Lee, and Hyeon Lee. 2025. OASIS: Outlier-Aware KV Cache Clustering for Scaling LLM Inference in CXL Memory Systems. *IEEE Computer Architecture Letters* (2025).
 - [26] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
 - [27] Llama Team. 2024. The Llama 3 Herd of Models. *CoRR abs/2407.21783* (2024). *arXiv:2407.21783* doi:10.48550/ARXIV.2407.21783
 - [28] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.
 - [29] Guanchu Wang, Zirui Liu, Zhimeng Jiang, Ninghao Liu, Na Zou, and Xia Hu. 2023. Division: memory efficient training via dual activation precision. In *International Conference on Machine Learning*. PMLR, 36036–36057.
 - [30] Darrell Williamson. 2012. *Discrete-time signal processing: an algebraic approach*. Springer Science & Business Media.
 - [31] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024*. OpenReview.net. <https://openreview.net/forum?id=NG7sS51zVF>
 - [32] Yi Xiong, Hao Wu, Changxu Shao, Ziqing Wang, Rui Zhang, Yuhong Guo, Junping Zhao, Ke Zhang, and Zhenxuan Pan. 2024. Layerkv: Optimizing large language model serving with layer-wise kv cache management. *arXiv preprint arXiv:2410.00428* (2024).
 - [33] Kai Xu, Minghai Qin, Fei Sun, Yuhao Wang, Yen-Kuang Chen, and Fengbo Ren. 2020. Learning in the frequency domain. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 1740–1749.
 - [34] Bin Yang, Qiuyu Leng, Jun Zeng, and Zhenhua Wu. 2025. CacheClip: Accelerating RAG with Effective KV Cache Reuse. *arXiv preprint arXiv:2510.10129* (2025).
 - [35] Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. 2025. KVLink: Accelerating Large Language Models via Efficient KV Cache Reuse. *CoRR abs/2502.16002* (2025). *arXiv:2502.16002* doi:10.48550/ARXIV.2502.16002
 - [36] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, 2369–2380. doi:10.18653/V1/D18-1259
 - [37] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*. 94–109.
 - [38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1–5, 2023*. OpenReview.net. https://openreview.net/forum?id=WE_vluYUL-X
 - [39] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 11608–11620.
 - [40] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.
 - [41] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 193–210.